

Computer Store Project User Manual

1. Project Overview

This project is a small computer store web application combining:

- A shopper storefront for browsing products and managing a cart.
- A customer repair portal for booking and tracking repair tickets.
- A staff backoffice for repair ticket management.
- An admin panel for catalog and product image management.

It uses a vanilla HTML/CSS/JavaScript frontend with a Node.js + Express backend and SQLite for persistence. The server database access layer is built with the `better-sqlite3` Node module.

2. Folder Structure

```

```
Laptop sale/
??? account.html
??? account.js
??? customer-auth.js
??? cart.html
??? cart.js
??? index.html
??? product.html
??? product-page.js
??? repair-book.html
??? my-repairs.html
??? styles.css
??? site.js
??? server/
? ??? index.js
? ??? db.js
? ??? auth.js
? ??? repairs.js
? ??? notify.js
? ??? upload.js
??? admin/
? ??? index.html
? ??? admin.js
? ??? admin.css
??? backoffice/
? ??? index.html
? ??? backoffice.js
??? data/
```

? ??? store.db  
? ??? uploads/  
??? package.json  
??? README.md  
` ` `

## Main components

- `account.html` / `account.js` ? customer login, repair portal, ticket list, notifications.
- `customer-auth.js` ? shared customer auth helpers and API wrapper.
- `backoffice/` ? staff repair management interface.
- `admin/` ? admin product management interface.
- `server/` ? backend API, data access, repair logic, auth, and uploads.
- `data/` ? persistence storage for the SQLite DB and uploaded images.

---

## 3. What Each Folder Contains

### Root-level pages

- `index.html` ? shop home.
- `cart.html` ? cart interface.
- `product.html` ? product detail page.
- `repair-book.html` ? request a repair.
- `my-repairs.html` ? view repair tickets.
- `account.html` ? account login/register and repair dashboard.

### `admin/`

- Admin product catalog management.
- Add/edit/delete products and change stock or price.
- Upload product images.

### `backoffice/`

- Staff-facing management of repair tickets.
- Technician workflow and calendar.

### `server/`

- `index.js` ? Express app entry point and route registration.
- `db.js` ? SQLite database functions and queries.
- `auth.js` ? authentication for customers, staff, and admins.
- `repairs.js` ? repair ticket creation, listing, and update business logic.
- `notify.js` ? email notification helper.
- `upload.js` ? product image upload handling.

---

## 4. How the Application Starts

## Setup steps

1. Run ``npm install``.
2. Copy ``.env.example`` to ``.env``.
3. Set your environment values, such as ``PORT``, ``JWT_SECRET``, ``ADMIN_USERNAME``, ``ADMIN_PASSWORD``.
4. Start the app with ``npm start``.

## Default URLs

- ``http://localhost:3000`` ? store homepage.
- ``http://localhost:3000/account.html`` ? customer login/register and repair portal.
- ``http://localhost:3000/repair-book.html`` ? repair booking form.
- ``http://localhost:3000/backoffice/`` ? staff backoffice.
- ``http://localhost:3000/admin/`` ? product administration panel.

---

## 5. Main User Roles

### Customer

- Browse products.
- Create an account.
- Add products to cart.
- Book repairs.
- View repair ticket status and updates.

### Staff

- Log in to  ``/backoffice/``.
- View and update repair tickets.
- Add technician notes, ETA, schedule, and note parts used.

### Admin

- Log in to  ``/admin/``.
- Manage the product catalog.
- Upload product images.
- Adjust prices and stock.

---

## 6. Backend API Summary

### Public API

- ``GET /api/products`` ? list products.
- ``GET /api/products/:id`` ? product details.
- ``GET /api/categories`` ? valid product categories.
- ``GET /api/settings`` ? store settings.

## Customer API

- `POST /api/customer/register` ? register a new customer.
- `POST /api/customer/login` ? customer login.
- `POST /api/repairs` ? create repair ticket.
- `GET /api/repairs/mine` ? list customer's repair tickets.
- `GET /api/repairs/mine/:ticketId` ? ticket details.

## Admin / Staff API

- `POST /api/auth/login` ? staff/admin login.
- `POST /api/products` ? create a product.
- `POST /api/products/:id/image` ? upload a product image.
- `PATCH /api/products/:id/price` ? update product price.
- `DELETE /api/products/:id` ? remove a product.
- `PATCH /api/repairs/:id` ? update a repair ticket.

---

## 7. How Data is Stored

- `data/store.db` ? SQLite database file.
- `data/uploads/` ? uploaded product image files.
- The database stores products, customers, users, repair tickets, repair updates, and cart contents.

---

## 8. Repair Workflow

### Customer side

1. Customer books a repair via `/repair-book.html`.
2. The frontend posts to `/api/repairs`.
3. The backend creates a ticket and saves it in SQLite.
4. The customer views the ticket list in `account.html`.
5. Ticket updates are visible in the repair details and notifications.

### Staff side

1. Staff update repair ticket status in `/backoffice/`.
2. The backend records updates and customer-visible notes.
3. Optional notification logic can email customers or log updates.

---

## 9. Architecture Diagram

### Runtime architecture

```text

[Browser Client]

```

?? public pages
?? customer portal
?? staff backoffice
?? admin panel
  |
  | HTTP / Fetch API
  v
[Express Server] server/index.js
?? public routes
?? customer routes
?? staff/admin routes
?? upload/notify helpers
  |
  v
[SQLite Database] data/store.db
  |
  v
[Uploads] data/uploads/
` ` `

```

Component flow

```

` ` `text
Customer Browser
?? account.html
?? repair-book.html
?? my-repairs.html
?? customer-auth.js
  |
  v
Backend API
?? /api/customer/register
?? /api/repairs
?? /api/repairs/mine
?? db.js / repairs.js
` ` `

---

```

10. Useful Files

- `README.md` ? quick start and summary.
- `package.json` ? install and start scripts.
- `server/index.js` ? Express entry point.
- `server/db.js` ? SQLite helpers.
- `server/repairs.js` ? repair logic.
- `account.html` / `account.js` ? customer repair portal.

- `customer-auth.js` ? auth and API wrapper.
- `admin/` ? product management UI.
- `backoffice/` ? staff workflow UI.

11. Notes

- The app can be run locally with Node and a `.env` file.
- Uploads are stored in `data/uploads` and the database in `data/store.db`.
- Customer auth uses JWT stored in `localStorage` for browser sessions.
- Staff/admin auth is handled by middleware in `server/auth.js`.

12. Recommended Next Steps

- Add explicit notification endpoint and message storage.
- Implement full end-to-end repair ticket tests.
- Add stronger auth protections for staff/admin routes.
- Add a README entry inside `user manual` for future documentation.